

ENSEA

Beyond Engineering

Rapport de Projet

Dassault UAV Challenge - Drone Infinity

Compte rendu final

3^{ème} année MSC

FLANDIN Elio
DUGARD Kevin
Tristan Noterman

Rapport de projet - Drone Infinity

Sommaire

α - Présentation - Objectif	3
I - Dassault UAV Challenge	3
II - Notre projet	3
2.1 - Notre équipe : répartition des tâches	3
2.2 - Ce que nous devons concevoir	4
2.3 - Cahier des charges	5
β - PCB	6
I - Avancée	6
II - Ce qu'il reste à faire	7
III - Pourquoi le PCB n'a pas abouti	8
γ - La nécessité des données	9
I - Quelles données ?	9
1.1 - Accélération	9
1.2 - Gyroscope	11
1.3 - Position GPS	13
1.4 - Exemple d'un capteur en plus : roue codeuse	15
II - Communication de ces données	15
2.1 - Données sur PuTTY	15
2.2 - Données sur l'écran	16

δ - Contrôle des Servomoteurs	17
I - Explication	17
II - Côté Ordinateur : Lecture de la manette et trame binaire	17
2.1 - Lecture de la manette : pygame	17
2.2 - Structure de la trame binaire	18
2.3 - Calcul et insertion du checksum côté Python	19
2.4 - Côté STM32 : Réception UART et décodage	19
2.4.1 - Configuration matérielle (.ioc)	19
2.4.2 - Réception en interruption	20
2.4.3 - Vérification de la trame	20
2.5 - Génération des signaux PWM pour les servos	20
2.6 - Validation de la PWM	21
2.7 - Tentative de pilotage d'un ESC + moteur brushless	21
2.8 - Notions de stabilisation à l'aide d'un PID et de l'IMU	22
ε - Asservissements	23
I - Asservissement des servomoteurs	23
II - Asservissement du moteur brushless	26
ζ - Un module en miniature ?	27
I - Commande du brushless par joystick	27
II - Initialisation du pilotage	27
III - Mode Failsafe	28
3.1 - Gestion de l'écran	28
3.1.1 - fonts.c	28
3.1.2 - sh1106.c	28
3.1.3 - oled_display.c	30
3.2 - Joystick et moteur	30
3.2.1 - Buzzers	30
3.2.2 - Contrôle du moteur	31
3.2.3 - Échange de données en bluetooth	32
3.2.4 - Feature de sécurité	34
η - Conclusion	35

α - Présentation - Objectif

I - Dassault UAV Challenge

Tout part d'ici : Le **Dassault UAV Challenge**. En tout début d'année l'un de nos professeurs nous propose de participer à ce dernier et d'en faire le sujet de notre projet. Mais avant d'être un projet il s'agit d'une compétition d'excellence organisée par **Dassault Aviation**, une compétition qui est rapidement devenue un rendez-vous incontournable pour les étudiants intéressés par l'aéronautique et les systèmes autonomes (ce qui colle donc avec notre spécialité). Ce défi propose aux futurs ingénieurs de sortir des salles de cours pour se confronter à une problématique industrielle concrète : la conception et le pilotage autonome d'un drone (aussi appelés UAV pour *Unmanned Aerial Vehicles*) au sein d'un environnement dynamique et exigeant.

En favorisant l'esprit d'équipe, la résilience face aux aléas techniques et la créativité, le Dassault UAV Challenge n'est pas une simple course de drones. C'est un véritable tremplin professionnel qui prépare la nouvelle génération d'ingénieurs à répondre aux enjeux de la défense et de l'aviation civile de demain.

Alors intéressés par de nouveaux challenges, nous avons décidé de monter une équipe de trois personnes et d'essayer d'y participer.

II - Notre projet

Partons donc sur la base de ce challenge, mais comme vous le comprendrez bientôt, nous allons nous en détacher assez rapidement. Ce dernier est d'ailleurs disponible sur notre github : [dépot github](#).

2.1 - Notre équipe : répartition des tâches

Comme annoncé précédemment nous sommes donc une équipe de trois étudiants en troisième année d'école d'ingénieur spécialisé en électronique et informatique. Dans notre cas, nous nous spécialisons en mécatronique et systèmes complexes.



Figure 1 - Notre équipe

Détaillons maintenant ce que chacun à réaliser pour ce projet (que chacun détaillera par la suite dans diverses parties). Commençons par Elio qui gère tout ce qui était réalisation d'une carte électronique, mesures, et asservissement, un peu de software mais aussi de hardware. Ensuite, Tristan qui s'occupe de deux grandes parties majeures : la motorisation et tout le côté puissance de notre projet, il est celui qui permettra à notre drone d'être contrôlé à distance. Et enfin, Kevin qui s'occupe à la fois du côté "managérial" du projet mais aussi d'un peu de software suite aux divers problèmes rencontrés.

Mais depuis tout à l'heure nous parlons de ce que chacun a dû faire sans même savoir quels étaient nos objectifs, parlons en plus concrètement maintenant.

2.2 - Ce que nous devons concevoir

Notre projet est simple dans le nom : un drone infini (d'où le nom *Infinity*). En nous basant sur Zephyr, le drone-satellite développé par Airbus, nous avons voulu le reproduire, à notre échelle et avec nos moyens.



Figure 2 - Drone-satellite Zéphyr d'Airbus

Ce véhicule a une particularité que l'on aperçoit instantanément sur la figure ci-dessus : il est recouvert de panneaux solaires. Le but de ce dernier est donc de pouvoir "voler" le plus longtemps possible.

Qu'allons nous donc faire ? L'objectif est simple, une fois de plus, dans les mots : un drone qui plane le jour et vole la nuit. Expliquons cela. Le jour notre drone ne vole pas vraiment, il plane c'est à dire qu'il utilise l'énergie qui est stockée dans une batterie seulement pour gérer sa direction et non son altitude (qui donc baisse petit à petit), mais avec l'effet de portance le but est de lui faire parcourir la plus grande distance possible tout en perdant le moins d'altitude possible. Et pendant cela, l'ensemble des panneaux solaires dont il est recouvert recharge sa batterie. Une fois la nuit tombée (et donc que nos panneaux solaires ne produisent plus d'électricité), il vole. Il active donc son moteur principal afin de regagner de l'altitude jusqu'à atteindre celle de référence.

Pour cela, on se retrouve donc avec divers capteurs (pression, gyroscope et accéléromètre entre autres) afin de pouvoir nous repérer et le stabiliser. Notre drone n'aura donc pas une forme de drone que l'on peut retrouver (quadrimoteur en forme de croix) mais bien d'un planeur, d'une aile en quelque sorte.

Mais c'est maintenant qu'il est important de préciser un élément : ce projet est irréalisable à notre échelle pour des tas de raisons. L'une d'entre elles est la taille de notre projet. En effet, notons que Zéphyr a une envergure de 22.5m et il nous serait impossible de concevoir quelque chose de la même taille. De plus, il vole à haute altitude afin de pouvoir optimiser la recharge par panneaux solaires et ne pas être dérangé par les nuages, ce qui est impossible pour nous.

C'est pourquoi nous avons assez rapidement décidé d'abandonner le Dassault UAV Challenge (aussi parce que l'on manquait de beaucoup d'informations et certaines nous étaient parvenues bien trop tardivement). Mais nous ne l'abandonnons pas complètement : avec le peu d'heures dont nous disposons, il nous est impossible de réaliser un véhicule concret marchant parfaitement. Nous avons donc pris la décision de démarrer ce projet. Par démarrer nous entendons le fait que, l'on espère, les élèves de l'année prochaine dans notre filière le reprendront afin de l'avancer (voir de le terminer). Et peut être, qui sait, ces derniers participeront au Dassault UAV Challenge. A la fin de ce projet, nous n'aurons donc pas un drone volant mais "seulement" quelques éléments que les prochaines pourront (et nous leur conseillons) utiliser pour reprendre ce projet.

2.3 - Cahier des charges

Maintenant que nous connaissons l'objectif de ce projet, allons davantage dans les détails et parlons du cahier des charges que nous avons fixé, les exigences que nous attendons pour notre véhicule. Ce dernier sera d'ailleurs à compléter par les élèves reprenant le projet.

ID	Intitulé / Fonction	Exigence technique	Précision	Priorité	Contraintes
ENER1	Stocker de l'énergie	Batterie LiPo	4S, 330mAh	Haute	Calcul de poids
ENER2	Recharge	Panneaux solaires	Production : ...	Moyenne	Calcul de surface
GLID	Planer (jour)	Vol plané sans propulsion	x	Haute	Optimisation de la durée
MOT1	Voler (nuit)	Propulsion motorisée	x	Haute	Calcul d'aérodynamisme
MOT2	Commande Moteur Brushless	Pilotage ESC -> PF070253	x	Haute	Temps de réponse
CONT	Contrôle à l'aide d'une manette	Liaison USB	Peu de latence, forte portée	Haute	Failsafe en cas de perte de signal
RAD	Recevoir des informations	Liaison radio bidirectionnelle	Peu de latence, forte portée	Moyenne	Portée
CAPT1	Mesure orientation	GY-91	Précision importante	Haute	Communication I²C
CAPT2	Mesurer accélération	GY-91	Précision importante	Moyenne	Communication I²C
CAPT3	Positionnement global	GPS GY-NEO6MV2	Rafraichissement rapide, précision ok	Moyenne	Antenne de bonne qualité
CAPT4	Gestion batterie	INA226AQDGSRQ1	x	Moyenne	Envoyer la valeur
SERV1	Contrôle des deux servomoteurs	PF070256	Précision importante	Moyenne	Asservi
IHM	Tenir l'utilisateur au courant	Ordinateur + Écran	x	Faible	Choix des informations
SAFE	Sécurité fail safe	Perte radio / Batterie faible	Atterrissage d'urgence	Critique	Consommer le moins de puissance
CAM	Récupérer des photos	Camera OV7670	x	Faible	Stockage / Envoi

Figure 3 - Cahier des charges

Et dès maintenant parlons de ce que nous avons complété, ce qui n'a pas été fini sans détailler (les détails viendront par la suite). Pour cela, rien de plus clair qu'un tableau résumant l'avancée de ce projet, l'avancée dans ce cahier des charges.

Validée	Partiellement validée	Pas validée
CONT RAD CAPT1 CAPT2 SERV1 IHM	MOT2 CAPT3 SAFE	ENER1 ENER2 GLID MOT1 CAPT4 CAM

Figure 4 - Avancée dans le cahier des charges à la fin du projet

Maintenant que tout est clair, nous pouvons commencer enfin à aller davantage dans les détails, en commençant par la réalisation d'une carte électronique : le PCB.

β - PCB

I - Avancée

On retrouve dessus en rouge tout ce qui concerne la partie puissance et en blanc la partie commande.

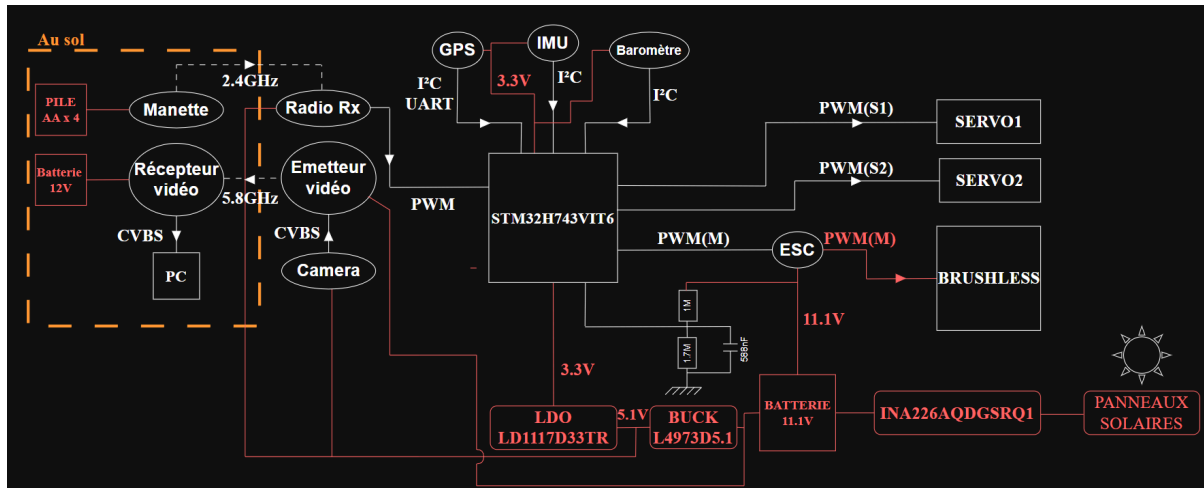


Figure 5 - Schéma d'architecture de notre projet

Nous avons, dans un premier temps, envisagé de concevoir puis de faire fabriquer une carte électronique dédiée à notre projet, afin d'obtenir un système plus optimisé, plus compact et plus léger qu'un assemblage de modules séparés. Cette approche aurait également permis de mieux maîtriser l'implantation des composants, le routage, la gestion de l'alimentation et la fiabilité globale du drone. Nous avons donc commencé par établir la liste des fonctions essentielles à intégrer sur le PCB, puis par sélectionner les composants principaux.

Tout d'abord, il était nécessaire d'abaisser la tension de la batterie du drone (11,1V) vers une tension intermédiaire stable de 5,1 V. Pour cela, nous avons choisi un convertisseur abaisseur (buck) L4973D5.1. Cette alimentation 5,1 V devait initialement servir à alimenter des éléments relativement consommateurs comme la caméra et l'émetteur vidéo. Cependant, la majorité des circuits numériques du projet fonctionnant en 3,3 V, une seconde conversion était indispensable : nous avons donc prévu un régulateur linéaire (LDO) LD1117D33TR afin de générer le 3,3 V à partir du 5,1 V, en garantissant une tension propre et compatible avec les capteurs et le microcontrôleur.

Nous avons également prévu l'intégration de plusieurs connecteurs afin de rendre la carte directement exploitable dans le drone : deux connecteurs pour les servomoteurs, un connecteur pour le driver/ESC, ainsi qu'un connecteur dédié à la caméra. En parallèle, des emplacements spécifiques ont été réservés pour les capteurs nécessaires au fonctionnement du système : un module GPS TESEO-LIV3R, un baromètre MPL3115A2ST1, une centrale inertielle IMU LSM6DSRXTR, ainsi qu'un capteur courant-tension INA226AQDGSRQ1, utilisé pour mesurer en temps réel la puissance délivrée par les panneaux solaires et suivre l'évolution énergétique du système. Enfin, nous avons prévu d'intégrer un microcontrôleur STM32H743VIT6, chargé de centraliser l'acquisition des capteurs, le traitement et la commande des différents actionneurs.

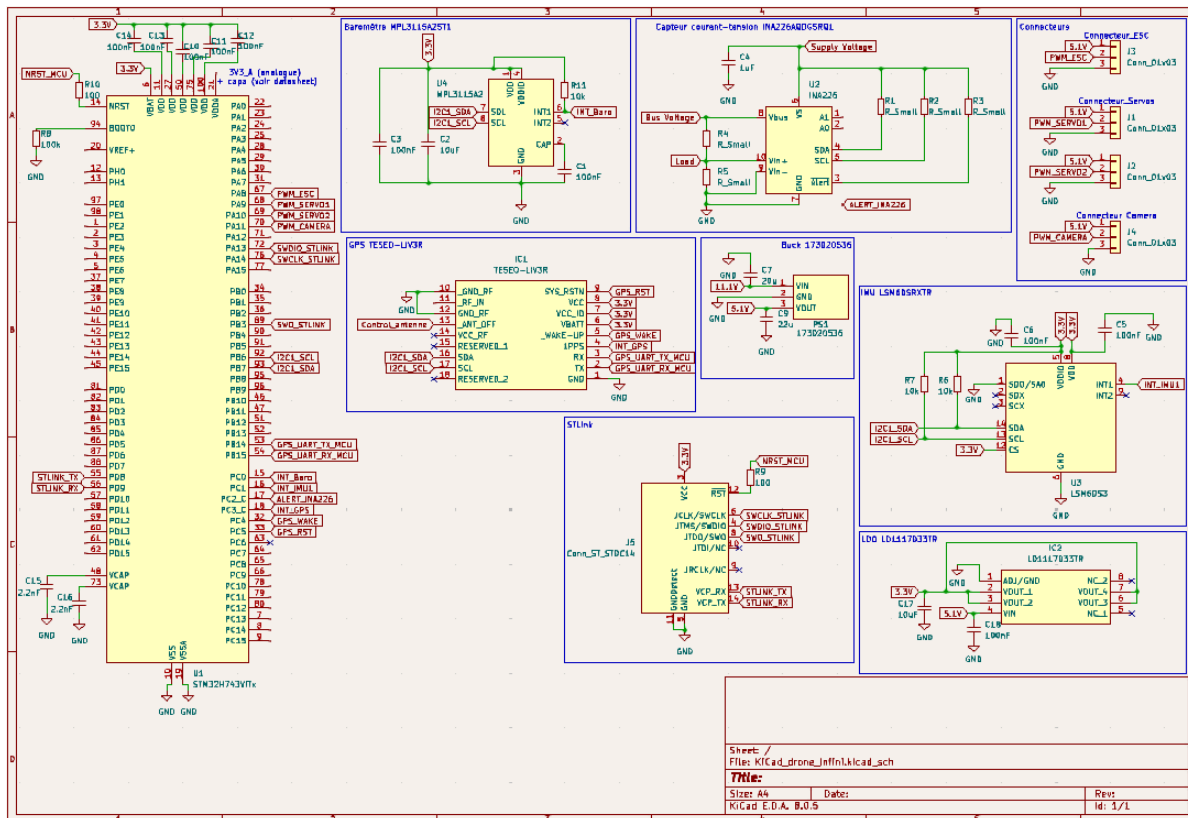


Figure 6 - Schématique KiCad du notre PCB envisagé

II - Ce qu'il reste à faire

À l'heure actuelle, il nous manque encore un PCB dédié à la manette de commande du drone. Cette carte aurait dû constituer l'interface principale entre le pilote et l'appareil : elle devait permettre à la fois de générer les signaux de commande nécessaires pour piloter les servomoteurs (afin d'orienter le drone et contrôler sa trajectoire) et de gérer la consigne de puissance du moteur arrière, via une commande adaptée vers l'ESC. Ce PCB aurait aussi pu nous servir d'affichage pour le pilote.

Pour le moment, faute de cette carte spécifique, nous utilisons uniquement une manette du commerce, qui nous permet de commander les servomoteurs de manière simple, mais qui ne répond pas complètement aux objectifs initiaux du projet, notamment en termes d'intégration, de personnalisation des fonctionnalités et d'optimisation du système de commande.

Il nous reste également à finaliser le PCB que nous avons commencé à concevoir sur KiCad. À ce stade, le schéma et l'architecture générale ont été initiés, mais plusieurs étapes essentielles restent à réaliser pour obtenir une carte réellement fonctionnelle : terminer la conception, effectuer le placement des composants puis le routage complet des pistes.

Une fois le routage validé, il faudra ensuite commander la carte, puis procéder à la soudure des composants, avant de pouvoir démarrer une phase de tests et de validation en conditions réelles.

III - Pourquoi le PCB n'a pas abouti

Malgré le travail de conception que nous avons déjà fait sur KiCad, le PCB n'a pas pu être finalisé et fabriqué dans le cadre de ce projet, principalement à cause des contraintes de planning. En pratique, la réalisation d'une carte complète implique la finalisation du routage, les vérifications électriques puis le délai de production et de livraison. À cela s'ajoutent le risque d'une erreur de câblage, une empreinte incorrecte ou un choix de composant à ajuster peut imposer une nouvelle version de carte, ce qui augmente encore les délais. Dans notre cas, le temps restant sur le projet ne permettait pas de garantir la réception d'un PCB fonctionnel avant la fin du projet, ni d'assurer une phase de tests et de validation suffisante en conditions réelles.

Nous avons donc fait le choix pragmatique de commander et d'utiliser des modules prêts à l'emploi intégrant déjà ces composants (capteurs, télécommande, etc.). Cette solution nous a permis de continuer le développement et les essais sans dépendre des délais de fabrication, tout en sécurisant l'avancement du projet et en conservant la possibilité, à terme, de revenir vers une intégration complète sur PCB lors de la reprise de ce projet.

N'ayant donc pas de PCB dédié, nous allons développer et tester le code des différents modules sur nos propres cartes STM32, en ajoutant des liaisons de communication adaptées : des interfaces UART série pour certains échanges, ainsi que le bus I²C lorsque cela est nécessaire pour communiquer avec nos différents capteurs.

γ - La nécessité des données

Je pense que personne ne sera surpris si nous annonçons que pour ce type de véhicule l'échange de données est plus que primordial, traitons donc cela dès maintenant.

I - Quelles données ?

Avant même de commencer cette partie, précisons quelque chose : les données que nous obtenons ne sont très probablement pas suffisantes, nous comptons donc sur nos successeur pour en rajouter. Il peut être intéressant dans un premier temps de rajouter quelques mesures intéressantes (avec un tube de pitot par exemple) mais aussi de doubler certaines (IMU et gyroscope par exemple), ce qui est fréquent dans le monde des véhicules volants.

1.1 - Accélération

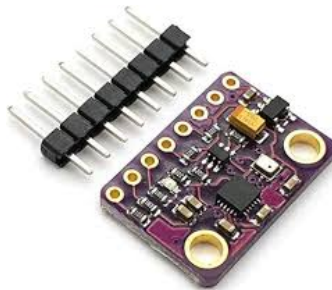


Figure 7 - Module IMU GY-91

L'acquisition des données d'accélération est réalisée à l'aide du capteur inertiel **MPU9250**, intégré au module GY-91, et communiquant avec le microcontrôleur via le bus **I²C**. Ce capteur fournit les accélérations selon trois axes orthogonaux (X, Y et Z), exprimées sous forme de valeurs numériques brutes codées sur 16 bits signés.

Commençons par l'implémentation des defines :

```
#define BMP280_ADDR    0x76
#define BMP280_ID_REG  0xD0

#define MPU9250_ADDR    0x68
#define MPU_WHO_AM_I    0x75
#define MPU_PWR_MGMT_1  0x6B
#define MPU_ACCEL_CONFIG 0x1C
#define MPU_GYRO_CONFIG  0x1B
```

Figure 8 - Déclaration des defines

Puis l'implémentation des variables globales :

```
BMP280_CalibData bmp_calib;
int32_t t_fine;

float gyro_offset_x = 0.0f;
float gyro_offset_y = 0.0f;
float gyro_offset_z = 0.0f;

float accel_offset_x = 0.0f;
float accel_offset_y = 0.0f;
float accel_offset_z = 0.0f;
```

Figure 9 - Déclaration des variables globales

Dans le programme, la lecture de l'accéléromètre est effectuée de manière cyclique à l'intérieur de la boucle principale afin d'obtenir des mesures en temps réel. À chaque itération, le microcontrôleur interroge le capteur par une transaction I²C en lecture à partir de l'adresse du registre de données de l'accéléromètre (adresse 0x3B), ce qui permet de récupérer simultanément les six octets correspondant aux composantes X, Y et Z. Ces octets sont ensuite assemblés deux par deux afin de reconstruire les valeurs brutes signées sur 16 bits pour chaque axe.

```
// === Lecture de l'accelero ===
uint8_t accel_raw[6];
int16_t ax_raw, ay_raw, az_raw;

HAL_I2C_Mem_Read(&hi2c2, MPU9250_ADDR << 1,
                 0x3B, I2C_MEMADD_SIZE_8BIT,
                 accel_raw, 6, HAL_MAX_DELAY);

ax_raw = (accel_raw[0] << 8) | accel_raw[1];
ay_raw = (accel_raw[2] << 8) | accel_raw[3];
az_raw = (accel_raw[4] << 8) | accel_raw[5];

// Conversion en g + Correction calibration
float ax = ax_raw / 16384.0f - accel_offset_x;
float ay = ay_raw / 16384.0f - accel_offset_y;
float az = az_raw / 16384.0f - accel_offset_z;
```

Figure 10 - Acquisition des données brutes de l'accéléromètre (dans le while(1))

Une fois les données brutes acquises, elles sont converties en valeurs physiques d'accélération exprimées en g (accélération de la pesanteur). Cette conversion repose sur le facteur d'échelle associé à la configuration du capteur, lequel est ici réglé sur une plage de mesure de ± 2 g, correspondant à une sensibilité de 16384 LSB/g. Chaque valeur brute est donc divisée par ce facteur afin d'obtenir une accélération normalisée. Afin d'améliorer la précision des mesures, une phase de calibration est réalisée au préalable : lorsque le capteur est maintenu immobile, plusieurs échantillons sont moyennés pour estimer les biais statiques propres à chaque axe (1000 échantillons). Ces offsets sont ensuite soustraits aux mesures instantanées, ce qui permet de corriger les erreurs systématiques dues aux imperfections internes du capteur et aux tolérances de fabrication.

```
// === Configuration du MPU9250 ===
// plage ±2g / ±250 dps
uint8_t accel_cfg = 0x00; // ±2g
uint8_t gyro_cfg = 0x00; // ±250 dps

HAL_I2C_Mem_Write(&hi2c2, MPU9250_ADDR << 1,
                  MPU_ACCEL_CONFIG, I2C_MEMADD_SIZE_8BIT,
                  &accel_cfg, 1, HAL_MAX_DELAY);

HAL_I2C_Mem_Write(&hi2c2, MPU9250_ADDR << 1,
                  MPU_GYRO_CONFIG, I2C_MEMADD_SIZE_8BIT,
                  &gyro_cfg, 1, HAL_MAX_DELAY);

// === Calibration ===
mpu9250_calibrate_gyro(1000);
mpu9250_calibrate_accel(1000);
```

Figure 11 - Configuration de l'accéléromètre et du gyroscope + calibration (une seule fois)

Lors de l'étape de calibration, nous réalisons d'abord une série de mesures sur les deux capteurs afin d'observer leur biais, puis nous calculons l'offset correspondant ; enfin, pour toutes les mesures suivantes, nous corrigeons les valeurs en soustrayant cet offset aux données acquises :

```

void mpu9250_calibrate_accel(uint16_t samples)
{
    int32_t ax_sum = 0, ay_sum = 0, az_sum = 0;
    uint8_t accel_raw[6];
    int16_t ax, ay, az;

    printf("Calibration de l'accéléromètre... Gardez le immobile!\r\n");

    for (uint16_t i = 0; i < samples; i++)
    {
        HAL_I2C_Mem_Read(&hi2c2, MPU9250_ADDR << 1,
                        0x3B, I2C_MEMADD_SIZE_8BIT,
                        accel_raw, 6, HAL_MAX_DELAY);

        ax = (accel_raw[0] << 8) | accel_raw[1];
        ay = (accel_raw[2] << 8) | accel_raw[3];
        az = (accel_raw[4] << 8) | accel_raw[5];

        ax_sum += ax;
        ay_sum += ay;
        az_sum += az;

        HAL_Delay(2);
    }

    accel_offset_x = (ax_sum / (float)samples) / 16384.0f;
    accel_offset_y = (ay_sum / (float)samples) / 16384.0f;
    accel_offset_z = (az_sum / (float)samples) / 16384.0f;
    // on ne retire pas 1g finalement car cela donnait une calibration fausse
    // (les 1g étaient retirés 2 fois)

    printf("Accel offsets [g]: %.3f %.3f %.3f\r\n",
           accel_offset_x, accel_offset_y, accel_offset_z);
}

```

Figure 12 - Fonction de calibration de l'accéléromètre

Les valeurs d'accélération corrigées représentent alors l'accélération réelle subie par le module L'acquisition continue et la correction systématique des données d'accélération constituent ainsi une étape fondamentale du système de stabilisation, fournissant une référence fiable et absolue indispensable à l'estimation de l'orientation du drone et au fonctionnement correct de la boucle d'asservissement par la suite.

1.2 - Gyroscope

Les données de vitesse angulaire sont acquises à l'aide du **gyroscope triaxial intégré au capteur MPU9250**, également présent sur le module GY-91. Comme pour l'accéléromètre, la communication entre le microcontrôleur et le capteur s'effectue via le bus **IPC**, ce qui permet une lecture fiable et synchrone des trois axes de rotation. Comme pour l'accéléromètre, le gyroscope fournit des mesures de vitesse angulaire selon les axes X, Y et Z. Dans le programme, ces données sont acquises de manière périodique à l'intérieur de la boucle principale afin de garantir une estimation continue et temps réel du mouvement angulaire du système.

```

// == Lecture du gyroscope ==
uint8_t gyro_raw[6];
int16_t gx_raw, gy_raw, gz_raw;

HAL_I2C_Mem_Read(&hi2c2, MPU9250_ADDR << 1,
                0x43, I2C_MEMADD_SIZE_8BIT,
                gyro_raw, 6, HAL_MAX_DELAY);

gx_raw = (gyro_raw[0] << 8) | gyro_raw[1];
gy_raw = (gyro_raw[2] << 8) | gyro_raw[3];
gz_raw = (gyro_raw[4] << 8) | gyro_raw[5];

// Conversion en deg/seconde + ajout des offsets
float gx = gx_raw / 131.0f - gyro_offset_x;
float gy = gy_raw / 131.0f - gyro_offset_y;
float gz = gz_raw / 131.0f - gyro_offset_z;

```

Figure 13 - Acquisition des données brutes du gyroscope (dans le while(1))

La lecture des données gyroscopiques est réalisée par une requête I²C à partir de l'adresse du premier registre de données du gyroscope (adresse 0x43). Cette opération permet de récupérer six octets consécutifs correspondant aux composantes X, Y et Z de la vitesse angulaire. Chaque paire d'octets est ensuite assemblée afin de reconstituer les valeurs brutes signées sur 16 bits pour chaque axe. Avant toute exploitation, le gyroscope est configuré via ses registres de contrôle afin de définir la plage de mesure souhaitée. Dans ce projet, une plage de ± 250 degrés par seconde a été sélectionnée, offrant une bonne résolution pour la stabilisation d'un drone à faible et moyenne dynamique. Cette configuration fixe le facteur d'échelle du capteur à 131 LSB par degré par seconde, valeur utilisée lors de la conversion des données brutes en unités physiques.

Une phase de calibration du gyroscope est indispensable pour garantir la précision des mesures. En effet, même lorsque le capteur est parfaitement immobile, le gyroscope présente des biais internes qui se traduisent par des vitesses angulaires non nulles. Afin de compenser ces erreurs systématiques, une fonction de calibration dédiée est exécutée au démarrage du programme. Durant cette phase, le capteur est maintenu immobile et un grand nombre d'échantillons est acquis (1000 échantillons, comme pour l'accéléromètre). Les valeurs mesurées sur chaque axe sont ensuite moyennées afin d'estimer les offsets statiques du gyroscope. Ces offsets, exprimés en degrés par seconde, sont mémorisés et systématiquement soustraits aux mesures instantanées lors du fonctionnement normal.

```
void mpu9250_calibrate_gyro(uint16_t samples)
{
    int32_t gx_sum = 0, gy_sum = 0, gz_sum = 0;
    uint8_t gyro_raw[6];
    int16_t gx, gy, gz;

    printf("Calibration du gyroscope... Gardez le immobile!\r\n");

    for (uint16_t i = 0; i < samples; i++)
    {
        HAL_I2C_Mem_Read(&hi2c2, MPU9250_ADDR << 1,
                        0x43, I2C_MEMADD_SIZE_8BIT,
                        gyro_raw, 6, HAL_MAX_DELAY);

        gx = (gyro_raw[0] << 8) | gyro_raw[1];
        gy = (gyro_raw[2] << 8) | gyro_raw[3];
        gz = (gyro_raw[4] << 8) | gyro_raw[5];

        gx_sum += gx;
        gy_sum += gy;
        gz_sum += gz;

        HAL_Delay(2); // 500 mesures par secondes
    }

    gyro_offset_x = (gx_sum / (float)samples) / 131.0f;
    gyro_offset_y = (gy_sum / (float)samples) / 131.0f;
    gyro_offset_z = (gz_sum / (float)samples) / 131.0f;

    printf("Gyro offsets [dps]: %.2f %.2f %.2f\r\n",
           gyro_offset_x, gyro_offset_y, gyro_offset_z);
}
```

Figure 14 - Fonction de calibration du gyroscope

Après calibration, les données brutes du gyroscope sont converties en vitesses angulaires réelles en divisant les valeurs numériques par le facteur d'échelle correspondant à la plage configurée, puis en appliquant la correction des offsets. Les vitesses angulaires ainsi obtenues constituent une mesure dynamique très réactive du mouvement du système. Elles sont utilisées pour intégrer les variations d'angle dans le temps et alimenter l'algorithme de fusion de capteurs, en complément des données issues de l'accéléromètre. Le gyroscope joue ainsi un rôle essentiel dans l'estimation de l'attitude du drone, en fournissant une information rapide et continue sur les rotations, indispensable au fonctionnement stable et précis de la boucle d'asservissement.

1.3 - Position GPS

La position de notre drone est, elle aussi, très importante. En effet, elle a plusieurs utilités :

- Enregistrer une position de référence à laquelle retourner en cas de problèmes.
- Permettre de localiser notre véhicule à tout moment.
- Faire du mapping par la suite (en combinaison avec la caméra).

Côté matériel, nous avons ici un GY-NEO 6M avec une antenne très bas de gamme. Le module GPS lui marche bien, mais le fait de prendre une antenne de faible qualité l'empêche de capter quelconque satellite. Ainsi, nous allons détailler ce que nous avons fait et les résultats obtenus même s'ils ne sont pas forcément positifs.

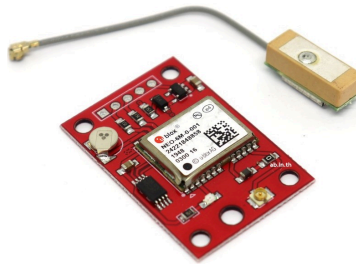


Figure 15 - Module GPS GY-NEO 6M

On retrouve peu de pins sur ce dernier, il n'est donc pas difficile de le câbler sur notre STM32. Puis il ne reste donc plus qu'à configurer notre carte pour cela, nous avons juste un récepteur et transmetteur USART afin de pouvoir échanger les données avec notre module. Mais ce n'est pas tout, il faut aussi brancher une autre pin à un GPIO : la pin PSS. Cette dernière n'est pas obligatoire mais améliore grandement la précision permettant de synchroniser avec une horloge. Et enfin les deux derniers pins concernent l'alimentation.

Puis il ne nous reste plus qu'à coder ce qu'il faut. Pour cela, nous créons le fichier `gps_neo6m.c` dans lequel nous retrouverons l'ensemble de nos fonctions concernant notre module. Nous ne mettrons dans ce rapport que les plus importantes et intéressantes (nous ne mettrons pas par exemple la fonction permettant la conversion de format NMEA en Degré Décimal, mais cela reste toujours disponible sur le github).

Commençons par l'implémentation de certaines valeurs importantes :

```
#define GPS_BUFFER_SIZE 256
static uint8_t rx_buffer[GPS_BUFFER_SIZE]; // Buffer circulaire (DMA ou IT)
static uint8_t rx_byte; // Byte temporaire pour IT
static uint16_t rx_index = 0;
static char line_buffer[GPS_BUFFER_SIZE]; // Ligne complète à parser
static volatile uint8_t line_received = 0; // Flag
static GPS_Data_t gps_data = {0};
static UART_HandleTypeDef *gps_huart;
```

Figure 16 - Déclaration des variables

Ici les commentaires suffisent comme précision, les noms restent assez clairs. Mais comme on le voit sur l'avant dernière ligne il y a un type de données que l'on ne voit jamais `GPS_Data_t`. Cela est normal, c'est une structure que nous avons introduite dans le `gps_neo6m.h`, structure que voilà :

```
typedef struct {
    float latitude;           /*!< Latitude in Decimal Degrees */
    char lat_dir;             /*!< N or S */
    float longitude;          /*!< Longitude in Decimal Degrees */
    char lon_dir;             /*!< E or W */
    float altitude;           /*!< Altitude in meters */
    float speed_kmh;          /*!< Speed in km/h */
    uint8_t satellites;       /*!< Number of satellites used */
    uint8_t fix_quality;       /*!< 0=Invalid, 1=GPS Fix, 2=DGPS Fix */
    char time[12];            /*!< UTC Time string (HHMMSS.SS) */
    uint8_t is_valid;         /*!< Flag to indicate valid data reception */
} GPS_Data_t;
```

Figure 17 - Structure GPS_Data_t

Maintenant que nos différentes valeurs sont initialisées, il faut initialiser l'ensemble de notre module, pour cela nous créons la fonction qui initialise simplement la réception par UART de notre gps : **GPS_Init**.

```
void GPS_Init(UART_HandleTypeDef *huart) {
    gps_huart = huart;
    // Démarre la réception caractère par caractère
    HAL_UART_Receive_IT(gps_huart, &rx_byte, 1);
}
```

Figure 18 - Initialisation de la reception UART

Puis nous créons un getter permettant de récupérer ces informations et de les stocker dans notre structure.

```
GPS_Data_t* GPS_GetData(void) {
    return &gps_data;
}
```

Figure 19 - Récupération et stockage des données

Et enfin, dernière fonction principale, celle qui sera appelée en boucle permettant de faire tourner le GPS et d'en obtenir des données. On retrouve dans cette dernière deux fonctions complexes permettant de trier les données et de récupérer les bonnes à partir de différents flag laisser dans les messages que ce dernier envoie : **GPS_Parse_GPGGA** et **GPS_Parse_GPRMC**.

```
void GPS_Process(void) {
    if (line_received) {
        // Dispatcher basé sur l'en-tête NMEA
        if (strncmp(line_buffer, "$GPGGA", 6) == 0) {
            GPS_Parse_GPGGA(line_buffer);
        }
        else if (strncmp(line_buffer, "$GPRMC", 6) == 0) {
            GPS_Parse_GPRMC(line_buffer);
        }
        else if (strncmp(line_buffer, "$GNSS", 6) == 0) {
            // Support pour les modules GNSS mixtes
            GPS_Parse_GPGGA(line_buffer);
        }
        line_received = 0;
    }
}
```

Figure 20 - Fonction principale de notre module

Désormais, notre GPS est prêt, il nous suffit d'appeler ce qu'il faut au bon endroit dans le `main.c` et le tour est joué, plus qu'à regarder ce que l'on reçoit par liaison série. Pour cela, nous utiliserons PuTTY.

Cependant, nous réussissons à établir la connexion avec notre GPS mais malheureusement ce dernier n'arrive à se connecter à aucun GPS, le problème vient donc bel et bien de l'antenne (qui comme annoncé précédemment reste bas de gamme).

1.4 - Exemple d'un capteur en plus : roue codeuse

Donnons un exemple intéressant de mesure supplémentaire à obtenir : la vitesse du moteur (et non du véhicule). Ceci permettrait principalement d'asservir le moteur en vitesse (l'ajout d'un capteur de courant serait optimal). De plus, cela n'est pas si compliqué à mettre en place grâce à nos cours et surtout aux séances de travaux pratiques de Systèmes d'Acquisition et de Commande où l'objectif est directement d'asservir un moteur en courant et en vitesse, il suffira donc de reprendre ce qui aura été fait pendant ces séances.

Au début nous avons eu comme idée d'ajouter aussi une caméra afin d'avoir un flux vidéo (ou de simple photo à la demande) mais suite à un problème de matériel cela peut aussi être intéressant d'en ajouter une.

II - Communication de ces données

Maintenant, nous avons tout un tas de données. Il est bien de les avoir mais il faut maintenant les communiquer à l'utilisateur. Pour cela, deux solutions : la liaison série avec l'ordinateur (mais cela implique de garder un ordinateur à portée de main pour le contrôler) ou un écran que l'on pourrait supposer ajouter à un nouveau PCB qui ferait office de manette. Ce fameux PCB serait composé de quelques joysticks, dudit écran ainsi que divers autres éléments (LED, Buzzers, module radio, etc.).

Ici, n'ayant pas de PCB pour une manette et notre manette (dont nous traiterons juste après) communique elle aussi en UART avec l'ordinateur, nous afficherons donc les différentes données que nous avons obtenues sur PuTTY. Mais nous montrerons tout de même par la suite que certaines données peuvent être affichées sur un écran dont nous avons déjà fait le code.

2.1 - Données sur PuTTY

Voici ainsi ce que l'on obtient sur PuTTY pour nos deux premiers jeux de valeurs (celui du GPS étant nulle, nous ne l'afficherons pas de nouveau).

```
===== Init Lecture GY-91 =====
Device found at 0x68
Device found at 0x76
BMP280 ID = 0x58
Temp = 24.01 °C | Press = 960.50 hPa
MPU9250 WHO_AM_I = 0x70
Calibration du gyroscope... Gardez le immobile!
Gyro offsets [dps]: 3.72 0.89 0.41
Calibration de l'accéléromètre... Gardez le immobile!
Accel offsets [g]: -0.057 0.022 -1.979
ROLL_REF=0.00 | ROLL=-0.00 | CMD=0.24
PITCH_REF=0.00 | PITCH=0.00 | CMD=-0.13
SERVO_L = 0.11 | SERVO_R = -0.37
ROLL_REF=0.00 | ROLL=0.86 | CMD=-2.75
PITCH_REF=0.00 | PITCH=0.70 | CMD=-2.24
SERVO_L = -4.99 | SERVO_R = 0.51
ROLL_REF=0.00 | ROLL=0.86 | CMD=-2.59
PITCH_REF=0.00 | PITCH=0.69 | CMD=-2.05
SERVO_L = -4.64 | SERVO_R = 0.53
ROLL_REF=0.00 | ROLL=-0.56 | CMD=1.96
PITCH_REF=0.00 | PITCH=1.18 | CMD=-3.63
SERVO_L = -1.67 | SERVO_R = -5.59
```

Figure 21 - Jeux de données de l'accéléromètre et du gyroscope sur PuTTY

2.2 - Données sur l'écran

Par exemple, si l'on avait une roue codeuse et ajouter deux trois features que nous détaillerons par la suite, voici ce que l'on pourrait obtenir sur un écran (le processus pour afficher ce qui se trouve ci-dessous sera expliqué ultérieurement).

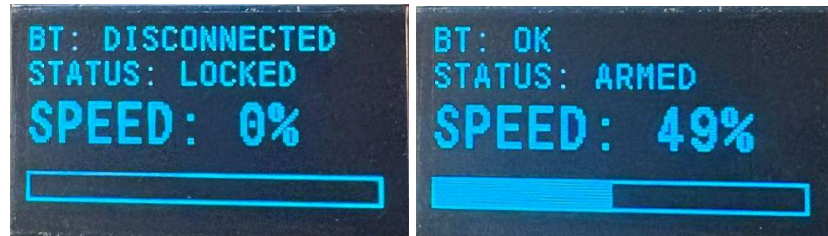


Figure 22 - Possibles données affichées sur un écran

Parlons très rapidement ici de ce que l'on voit d'afficher. Tout d'abord nous avons l'état de la connection bluetooth (qui est notre connection temporaire comme nous n'avons pas de module radio), le fait de savoir si notre drone est en veille (locked) ou en utilisateur (armed) et enfin la vitesse de ce dernier (ici il s'agit de la vitesse d'un moteur bas de gamme arduino, donc pas besoin de roue codeuse, même si cette vitesse n'est pas donc pas celle du moteur mais celle envoyée par la STM).

δ - Contrôle des Servomoteurs

Passons désormais à ce qui va nous permettre de gérer la direction et notre drone : deux servomoteurs (un sur chaque aile allant de haut en bas). Ces derniers sont reliés à des ailettes permettant, comme un vrai avion, de faire tourner notre UAV d'un côté ou de l'autre selon le choix de l'utilisateur, le tout en jouant avec l'impact sur flux d'air sur ce dernier.

I - Explication

L'objectif de cette partie est de réaliser une chaîne complète de commande entre : une manette de jeu tout public (type Xbox 360), un ordinateur exécutant un script Python, une carte STM32 ainsi que les fameux servos-moteurs (et, en option, un ESC et un moteur brushless).

L'idée générale (processus) est la suivante :

1. L'utilisateur agit sur les joysticks et boutons de la manette.
2. Un script Python lit ces entrées, les convertit en valeurs numériques normalisées, construit une trame binaire et l'envoie à la STM32 via un lien série UART.
3. La STM32 reçoit la trame, la decode, puis génère des signaux PWM à destination des servos.
4. La même logique de PWM est réutilisable pour piloter un ESC de moteur brushless avec un second joystick.

À plus long terme, ce système a vocation à être couplé avec une centrale inertielle (IMU) et un correcteur PID, afin de stabiliser automatiquement une plateforme ou un mini-drone, la manette donnant alors des consignes de roulis (roll), tangage (pitch), etc.

Ainsi, séparons les rôles : l'ordinateur fait l'interface "haut niveau" avec la manette et prépare les données. La STM32, elle, gère le temps réel, le décodage et la génération de signaux pour l'électronique de puissance (servos / ESC).

II - Côté Ordinateur : Lecture de la manette et trame binaire

2.1 - Lecture de la manette : pygame

Comme annoncé, pour lire la manette, nous allons utiliser un script Python. Mais pour cela, il faut installer quelques bibliothèques que voilà :

Nom	Fonction
<i>pygame</i>	Gestion des manettes / joysticks
<i>pyserial</i>	Communication série avec la STM32

Figure 23 - Bibliothèques python à installer

Au lancement du script, *pygame* qui est initialisé grâce à la fonction *pygame.init*. La manette détectée est ouverte *pygame.joystick.Joystick(0)* et un message de confirmation est affiché : "Manette détectée : Xbox 360 Controller".

Puis dans la boucle principale, on appelle `pygame.event.pump()` pour mettre à jour l'état de la manette, puis on lit les axes :

```
raw_roll    = joy.get_axis(0)    # Stick gauche, axe X
raw_pitch   = joy.get_axis(1)    # Stick gauche, axe Y
raw_yaw     = joy.get_axis(3)    # Stick droit, axe X
raw_throttle = joy.get_axis(4)   # Gâchette / autre axe
```

Figure 24 - Lecture des axes

Ici, chaque axe `get_axis` renvoie une valeur flottante dans comprise entre -1.0 et +1.0. Puis, ces valeurs brutes des axes sont converties dans une plage entière comprise entre -1000 et +1000, ce qui est plus adaptée à la suite du traitement.

```
CMD_MAX = 1000

def axis_to_int16(x: float) -> int:
    if x > 1.0: x = 1.0
    if x < -1.0: x = -1.0
    return int(x * CMD_MAX)
```

Figure 25 - Conversion des axes sur notre plage

Ensuite, il faut mapper. Nous le faisons donc pour roll et yaw. Mais attention, pour pitch, il est souvent inversé (sur la plupart des manettes, lorsque l'on pousse le joystick vers l'avant, on obtient une valeur négative), il faudra donc penser à mettre un signe négatif avant la commande de ce dernier. Voici ce que nous proposons :

```
pitch = axis_to_int16(-raw_pitch)
roll  = axis_to_int16(raw_roll)
yaw   = axis_to_int16(raw_yaw)
```

Figure 26 - Commande pitch (lacet), roll (roulis) et yaw (tangage)

2.2 - Structure de la trame binaire

Maintenant que nous pouvons récupérer les commandes de nos joysticks, il faut pouvoir les envoyer à notre STM32, pour cela, nous définissons un trame binaire de 11 octets par le biais d'une structure sur STM32 :

```
typedef struct __attribute__((packed)) {
    uint8_t stx;           // 0xAA = début de trame
    int16_t throttle;      // -1000 à 1000 (ou 0 à 1000)
    int16_t roll;
    int16_t pitch;
    int16_t yaw;
    uint8_t flags;         // bits de commande (ARM, etc.)
    uint8_t checksum;      // somme des octets précédents (mod 256)
} control_frame_t;
```

Figure 27 - Structure de la trame binaire

Ainsi, en mémoire (et sur la liaisons série), nous avons donc bien un total de 11 octets :

```

      STX (1 OCTET) : 0xAA
  THROTTLE (2 OCTETS, INT16 LITTLE ENDIAN)
      ROLL (2 OCTETS)
      PITCH (2 OCTETS)
      YAW (2 OCTETS)
      FLAGS (1 OCTET)
      CHECKSUM (1 OCTET)
  
```

Figure 28 - Trame binaire

2.3 - Calcul et insertion du checksum côté Python

Nous construisons donc désormais cette trame en Python grâce à une nouvelle bibliothèque : *struct* et sa fonction *struct.pack*.

```

def send_frame(ser, throttle, roll, pitch, yaw, flags):
    stx      = 0xAA
    payload = struct.pack('<BhhhhB', stx, throttle, roll, pitch, yaw, flags)
    checksum = sum(payload) & 0xFF
    frame    = payload + struct.pack('<B', checksum)
    ser.write(frame)
  
```

Figure 29 - Construction de la trame

Ici précisons un élément qui n'est pas laissé au hasard : *<BhhhhB*. Cette chaîne de caractère signifie en réalité "little-endian" soit 1 octet. Puis nous calculons la somme de l'ensemble de ces octets (sauf le checksum elle-même), puis on ne garde que le LSB.

Ensuite, la STM32 peut recalculer la même somme et vérifier que les deux coïncident. Afin de permettre la détection d'erreurs de transmission.

Précisons d'ailleurs que la boucle principale Python envoie une trame environ toutes les 20 ms (grâce à la fonction *time.sleep(0.02)* de la bibliothèque *time*), soit 50 Hz.

2.4 - Côté STM32 : Réception UART et décodage

Passons désormais de nouveau du côté de notre carte STM32 et observons la façon dont elle reçoit cette trame et comment la decode-t-elle. Mais pour cela, nous devons passer par différentes étapes avant.

2.4.1 - Configuration matérielle (.ioc)

Signal	Utilité
UART	Communication série avec l'ordinateur
TIM3	Génération des PWMs

Figure 30 - Configuration de l'.ioc

Puis nous configurons la liaison série avec les paramètres suivants : une vitesse de 115200 bauds, une longueur de mot de 8 bits, pas de parité et un bit de stop.

2.4.2 - Réception en interruption

Concernant la réception, nous allons l'effectuer à l'aide d'une interruption. Pour cela, nous utilisons la fonction de la bibliothèque *HAL* : *HAL_UART_Receive_IT(&huart2, &rx_byte, 1)*. Puis, à chaque octet reçu, une fonction de callback est appelée. On a donc une petite machine à états pour reconstituer la trame :

- *RX_WAIT_STX* : attente du byte 0xAA,
- *RX_READ_FRAME* : remplissage du buffer jusqu'à 11 octets,
- puis appel de *process_control_frame()*

2.4.3 - Vérification de la trame

Mais que fait notre fonction appelée au bout de cette machine à état ? Elle vérifie que le premier octet est bien 0xAA et elle recalcule le checksum. Si ce dernier est correct, on décode les champs, on met à jour les variables globales puis on affiche le tout dans la console (toujours grâce à PuTTY).

```
Frame OK: thr=1000 roll=-633 pitch=-32 yaw=0 flags=0x00
Frame OK: thr=1000 roll=-919 pitch=-72 yaw=0 flags=0x00
Frame OK: thr=1000 roll=-693 pitch=-216 yaw=0 flags=0x00
Frame OK: thr=1000 roll=-45 pitch=-470 yaw=0 flags=0x00
Frame OK: thr=1000 roll=467 pitch=-864 yaw=0 flags=0x00
Frame OK: thr=1000 roll=578 pitch=-815 yaw=0 flags=0x00
Frame OK: thr=1000 roll=580 pitch=-814 yaw=0 flags=0x00
Frame OK: thr=1000 roll=580 pitch=-814 yaw=0 flags=0x00
Frame OK: thr=1000 roll=565 pitch=-824 yaw=0 flags=0x00
```

Figure 31 - Affichage des données reçues par notre STM32

Et quand on bouge les joysticks, roll varie symétriquement de -1000 à +1000, de même pour pitch mais avec un signe inversé et throttle reste dans l'intervalle [0 ; 1000]. Quant aux flags, ils passent à 0x01 si on appuie sur un bouton (par exemple, le bouton A).

Ainsi, Ces trames confirment la bonne réception, le décodage et la vérification du checksum

2.5 - Génération des signaux PWM pour les servos

Désormais, nous recevons bien ce que l'on veut, il faut ainsi générer les PWM pour les servomoteurs à partir de ceci. Pour cela, on utilise TIM3 pour générer des signaux PWM à une fréquence de 50Hz (ce qui est standard pour les servos RC) avec pour paramètres principaux une fréquence d'horloge de 170MHz, un prescaler PSC de 169 et une période ARR de 19999. On retrouve ainsi bien notre période de 20ms et nous avons un tick par microseconde.

De plus, nous avons que nos servos attendent typiquement la période réglée ci-dessus et pour les impulsions :

- 1000µs : butée d'un côté
- 1500µs : position neutre
- 2000µs : butée de l'autre côté

De plus, comme le timer compte à 1MHz, on peut directement écrire une valeur en μs dans le registre CCR à l'aide de `__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 1500)` (le 1500 représente une impulsion de 1500 μs soit 1.5ms).

Ainsi, on utilise la valeur `cmd_roll` (reçue de la manette, qui est comprise entre -1000 et +1000) pour commander le servo. On veut de cette dernière obtenir une impulsion comprise entre 1000 μs et 2000 μs , on convertit ainsi l'échelle à l'aide d'une fonction affine.

2.6 - Validation de la PWM

En observant le servo, on a une impulsion de 1500 μs lorsque l'on envoie aucune trame (position neutre) et si l'on bouge le joystick horizontal correspondant au roll, le servomoteur pivote donc bien à gauche ou à droite. De plus, l'amplitude est cohérente avec les positions mécaniques extrêmes de notre servomoteur.

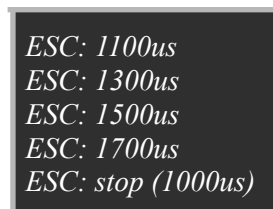
D'ailleurs, un oscilloscope permettrait aussi de vérifier que la largeur d'impulsion varie entre 1 ms et 2 ms.

2.7 - Tentative de pilotage d'un ESC + moteur brushless

Mais ce n'est pas tout, en plus des servos, nous avons tenté de piloter un ESC (Electronic Speed Controller) pour moteur brushless (celui qui nous permettrait de regagner de l'altitude), en réutilisant le même type de signal PWM. Notons qu'un ESC "type drone" se commande généralement comme un servo : avec un signal PWM à la fréquence de 50Hz ou de 400Hz et une impulsion de 1000 μs pour être au gaz minimum et une 2000 μs pour être au maximum.

Pour cela, nous avons raccordé l'ESC à une alimentation stabilisée de laboratoire, branché le fil de signal sur TIM3_CH2 (PB5), puis générer des séquences de type : 1000 μs pendant l'armement puis des paliers tels que 1100 μs , 1300 μs , 1500 μs , 1700 μs , etc.

Notre terminal affichait donc des messages comme :



```
ESC: 1100us
ESC: 1300us
ESC: 1500us
ESC: 1700us
ESC: stop (1000us)
```

Figure 32 - Terminal pendant les paliers de la PWM pour le moteur brushless

Ainsi, on observe comme résultat le fait que l'ESC émet les bips caractéristiques d'initialisation mais malheureusement le moteur ne se met pas à tourner, même à des consignes supérieures au minimum. Ainsi, plusieurs hypothèses possibles qu'on a noté :

- L'ESC peut attendre une séquence de calibration spécifique (par exemple : maintenir 2000 μs à la mise sous tension, puis 1000 μs).
- L'alimentation de labo ne reproduit peut-être pas exactement le comportement d'une LiPo (chutes de tension, courant instantané, etc.).
- Il y a peut-être un conflit entre la fréquence ou la forme du signal attendu et ce que nous envoyons.

Ainsi, nous ne sommes pas en capacité de contrôler notre moteur brushless avec notre manette, mais le code étant très bien avancé, il sera aisé de le reprendre afin de régler ce problème l'année prochaine.

2.8 - Notions de stabilisation à l'aide d'un PID et de l'IMU

En parallèle, nous avons donc essayé de travailler sur l'asservissement en position de notre drone à l'aide du module présenté dans la partie précédente. L'objectif étant d'obtenir les angles roll et pitch du système, afin de pouvoir fermer une boucle de régulation (PID) sur ces angles.

L'idée finale étant de combiner :

- Les références *roll_ref* / *pitch_ref* venant de la manette (via la trame)
- Les mesures *roll* / *pitch* issues de l'IMU
- Les correcteurs PID qui produisent des commandes pour les servos.

Ensuite, on mappe ces commandes (en degrés ou en valeurs normalisées) vers des signaux PWM de 1000–2000 µs, de la même façon que pour *cmd_roll* dans notre réalisation actuelle.

Mais par faute de temps, cette boucle complète n'a pas été intégrée et testée en conditions réelles, mais toute la "brique" IMU + PID est prête côté logiciel, et c'est de cette dernière que nous allons désormais discuter.

ε - Asservissements

I - Asservissement des servomoteurs

```
#define RAD_TO_DEG (57.2957795f)
#define DT_TO_SEC 1000.0f

#define PID_MAX 10.0f
#define ANGLE_DEADBAND 2.0f
```

Figure 33 - Déclaration des defines

```
float roll = 0.0f;
float pitch = 0.0f;

float alpha = 0.98f; // coefficient du filtre
uint32_t last_tick = 0;

typedef struct {
    float kp;
    float ki;
    float kd;

    float prev_error;
    float integral;
} PID_t;

PID_t pid_roll;
PID_t pid_pitch;

float roll_ref; // valeur de référence obtenues par la télécommande
float pitch_ref; // en degrés

float servo_left = 0.0f; // Valeurs de sorties
float servo_right = 0.0f; // pour la commande des servos
```

Figure 34 - Déclaration des variables globales

L'asservissement des servomoteurs constitue l'élément central du système de stabilisation développé dans ce projet. Il permet de transformer les informations issues des capteurs inertiels en commandes physiques agissant directement sur les gouvernes de l'aile volante. Cette chaîne de contrôle repose sur plusieurs étapes successives, illustrées par les figures associées aux fonctions logicielles implémentées. Dans un premier temps, les données d'accélération mesurées par l'accéléromètre sont exploitées afin d'estimer les angles d'attitude du système. La fonction *compute_roll_pitch* (Figure 36) calcule les angles de roulis (roll) et de tangage (pitch) à partir de la direction du vecteur gravité. Ces angles représentent l'orientation instantanée du drone par rapport à l'horizontale et constituent la mesure d'état utilisée pour la stabilisation.

```
void compute_roll_pitch(float ax, float ay, float az,
                       float *roll, float *pitch)
{
    *roll = atan2f(ay, az) * RAD_TO_DEG;
    *pitch = atan2f(-ax, sqrtf(ay*ay + az*az)) * RAD_TO_DEG;
}
```

Figure 35 - Fonction de calcul des angles de roulis et de tangage à partir des données de l'accéléromètre

Les angles mesurés sont ensuite comparés aux consignes d'angle issues de la télécommande, ce qui permet de calculer une erreur de position angulaire pour chaque axe. Cette erreur est traitée par un correcteur de type PID discret, implémenté dans la fonction `pid_update` (Figure 37). Le contrôleur PID calcule une commande corrective à partir de la combinaison pondérée de trois termes : un terme proportionnel, qui génère une correction proportionnelle à l'erreur instantanée, un terme intégral (désactivé dans la version actuelle), destiné à compenser les erreurs statiques, et un terme dérivé, qui anticipe les variations rapides de l'erreur et améliore la stabilité du système. Les gains associés à ces trois termes sont initialisés au démarrage du programme (Figure 38) et ont été choisis volontairement faibles afin d'assurer un comportement stable et progressif lors des premiers essais. Ces paramètres favorisent une réponse douce du système, adaptée à une structure mécanique légère pilotée par des servomoteurs.

```
float pid_update(PID_t *pid, float error, float dt)
{
    pid->integral += error * dt;

    float derivative = (error - pid->prev_error) / dt;
    pid->prev_error = error;

    return pid->kp * error
        + pid->ki * pid->integral
        + pid->kd * derivative;
}
```

Figure 36 - Fonction de calcul de la commande PID pour l'asservissement en attitude

```
// Gains PDI (à revoir après test grandeur nature)
pid_roll.kp = 0.6f;
pid_roll.ki = 0.0f;
pid_roll.kd = 0.02f;
pid_roll.prev_error = 0.0f;
pid_roll.integral = 0.0f;

pid_pitch.kp = 0.6f;
pid_pitch.ki = 0.0f;
pid_pitch.kd = 0.02f;
pid_pitch.prev_error = 0.0f;
pid_pitch.integral = 0.0f;
```

Figure 37 - Gain du PID (pour l'instant seulement Proportionnel Dérivé)

La sortie du correcteur PID correspond à une commande angulaire corrective exprimée en degrés, limitée par saturation afin d'éviter toute sollicitation excessive des servomoteurs. Cette commande est ensuite transformée en signaux exploitables par les servomoteurs grâce à la fonction `angle_to_pwm` (Figure 39). Cette fonction réalise une conversion linéaire entre la commande angulaire et la largeur d'impulsion PWM, en respectant la plage de fonctionnement de nos servomoteurs comprise entre 1000 µs et 2000 µs, avec une position neutre fixée à 1500 µs. Cette étape assure une correspondance cohérente entre la commande issue du correcteur et la position mécanique réelle des gouvernes.

```

uint16_t angle_to_pwm(float angle_deg)
{
    // Saturation à ±PID_MAX
    if (angle_deg > PID_MAX) angle_deg = PID_MAX;
    if (angle_deg < -PID_MAX) angle_deg = -PID_MAX;

    // Conversion angle → PWM
    // -PID_MAX → 1000 µs
    // 0 → 1500 µs
    // +PID_MAX → 2000 µs
    float k = 500.0f / PID_MAX;
    float pulse = 1500.0f + angle_deg * k;

    // Sécurité finale
    if (pulse < 1000.0f) pulse = 1000.0f;
    if (pulse > 2000.0f) pulse = 2000.0f;

    return (uint16_t)pulse;
}

```

Figure 38 - Fonction de conversion de la commande angulaire en signal PWM pour le pilotage des servomoteurs

Dans le cas spécifique de l'aile volante, les commandes de roulis et de tangage sont combinées par un mixage approprié afin de générer deux signaux distincts destinés aux servomoteurs gauche et droit. Ce mixage permet à chaque gouverne de contribuer simultanément aux mouvements de roulis et de tangage, conformément à l'architecture aérodynamique du drone.

```

// === Calcul roll / pitch ===
float roll_acc, pitch_acc;
compute_roll_pitch(ax, ay, az, &roll_acc, &pitch_acc);

// === Code du filtre ===
// Temps
uint32_t now = HAL_GetTick();
float dt = (now - last_tick) / DT_TO_SEC; // en secondes
last_tick = now;

roll += gx * dt; // Intégration du gyro
pitch += gy * dt;

roll = alpha * roll + (1.0f - alpha) * roll_acc; // Filtre complémentaire
pitch = alpha * pitch + (1.0f - alpha) * pitch_acc;

HAL_Delay(20);

float err_roll = roll_ref - roll;
float err_pitch = pitch_ref - pitch;

```

Figure 39 - Calcul du roll/pitch et du filtre complémentaire pour le calcul de l'erreur

```

// Pour ne pas corriger du bruit ou un angle trop faible
if (fabsf(err_roll) < ANGLE_DEADBAND)
    err_roll = 0.0f;

if (fabsf(err_pitch) < ANGLE_DEADBAND)
    err_pitch = 0.0f;

float roll_cmd = pid_update(&pid_roll, err_roll, dt);
float pitch_cmd = pid_update(&pid_pitch, err_pitch, dt);

if (roll_cmd > PID_MAX) roll_cmd = PID_MAX;
if (roll_cmd < -PID_MAX) roll_cmd = -PID_MAX;

if (pitch_cmd > PID_MAX) pitch_cmd = PID_MAX;
if (pitch_cmd < -PID_MAX) pitch_cmd = -PID_MAX;

// Mixage Servos
servo_left = pitch_cmd + roll_cmd;
servo_right = pitch_cmd - roll_cmd;

uint16_t servo_left_pwm = angle_to_pwm(servo_left);
uint16_t servo_right_pwm = angle_to_pwm(servo_right);

printf("ROLL_REF=%.2f | ROLL=%.2f | CMD=%.2f\r\n",
    roll_ref, roll, roll_cmd);
printf("PITCH_REF=%.2f | PITCH=%.2f | CMD=%.2f\r\n",
    pitch_ref, pitch, pitch_cmd);
printf("SERVO_L = %u us | SERVO_R = %u us\r\n",
    servo_left_pwm, servo_right_pwm);

```

Figure 40 - Utilisation de toute les fonctions et mixage des servomoteurs

L'ensemble de cette chaîne logicielle permet ainsi de maintenir le drone stable autour de la consigne demandée, tout en autorisant des changements d'orientation commandés par le pilote. L'asservissement mis en place garantit une stabilisation continue et progressive, constituant une base robuste pour les essais en conditions réelles et les évolutions futures du système de contrôle.

II - Asservissement du moteur brushless

Pour l'asservissement du moteur de puissance de notre drone, une approche d'asservissement similaire à celle étudiée lors du TP de système d'acquisition et de commande aurait pu être mise en œuvre pour le moteur brushless principal. En effet, un asservissement en cascade, composé d'une boucle interne de courant et d'une boucle externe de vitesse, constitue une architecture classique et robuste pour la commande de moteurs de puissance.

La mesure du courant moteur, réalisée via des capteurs de courant placés sur les phases ou sur le bus continu, permettrait de contrôler directement le couple électromagnétique produit par le moteur, garantissant ainsi une limitation efficace du courant et une protection des composants de puissance. La vitesse de rotation pourrait être estimée à partir des informations fournies par les capteurs Hall intégrés au moteur ou par des méthodes d'estimation sans capteur (sensorless), basées sur la mesure des tensions et courants de phase. La boucle de vitesse fournirait alors une consigne de courant en fonction de l'erreur entre la vitesse mesurée et la vitesse désirée.

Enfin, la commande de puissance serait assurée par une modulation de largeur d'impulsion (PWM) appliquée aux transistors de l'onduleur triphasé, synchronisée avec les mesures pour garantir une régulation stable et réactive. Une telle structure aurait permis d'obtenir une commande précise du moteur, une meilleure dynamique du drone et une intégration cohérente avec les lois de pilotage globales de l'aéronef.

ζ - Un module en miniature ?

Comme annoncé précédemment, nous avons eu un problème au niveau de notre moteur brushless et plus particulièrement au niveau de notre ESC. Ainsi, afin d'avoir à la fois un élément à démontrer lors de notre présentation mais aussi de voir comment réagit notre module suite à certaines features. Ainsi, sur une breadboard, nous avons effectué un petit montage que nous détaillerons en plusieurs parties.

I - Commande du brushless par joystick

Commençons par la base, le but étant de contrôler un moteur à l'aide d'un joystick. Alors nous avons pris un moteur arduino (qui n'a donc rien à voir côté puissance avec celui de notre drone), un joystick, une breadboard, quelques câbles et nous allons essayer cela.



Figure 41 - Moteur arduino

II - Initialisation du pilotage

Mais ce n'est pas tout, nous allons aussi ajouter le fait de pouvoir démarrer et arrêter notre drone (le fameux armed/locked) vu dans une figure précédente. Pour cela, nous allons instaurer une procédure (facilement modifiable par la suite) où il faudra rester appuyer un certain temps sur le joystick. Tant que cela n'est pas fait, il est impossible de prendre le contrôle de notre drone.

Nous ajoutons donc un buzzer et quelques LEDs afin d'annoncer à l'utilisateur quand le drone est utilisable ou non. Nous ajouterons en plus quelques messages sur la console UART et sur l'écran pour que l'utilisateur soit au courant en temps réel de l'état de commandabilité de son drone.

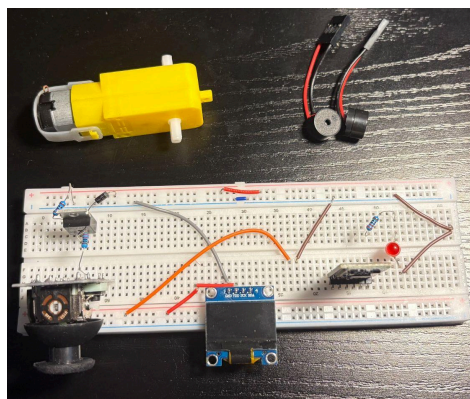


Figure 42 - Montage sur la breadboard

Précisons ici que nous avons retiré l'ensemble des câbles dupont afin de proposer une meilleure visibilité. On retrouve bien les buzzers, le moteur, le joystick, l'écran ainsi que le module bluetooth. On retrouve aussi quelques résistances afin de gérer certains aspects du courant et une diode de chute libre juste après le transistor (permettant le contrôle du moteur).

III - Mode Failsafe

Et enfin, nous ajoutons un mode Failsafe, un mode qui lorsqu'un problème est rencontré, le drone établit un protocole de sécurité. Dans l'optimal ce dernier retournera à la position initiale (obtenue par le GPS) lorsqu'un des problèmes suivants est rencontré : batterie faible, perte de signal avec la télécommande ou surchauffe.

Ainsi, nous avons ici besoin de plusieurs fichiers pour l'ensemble de nos composants (une fois de plus ces fichiers sont disponibles sur le github et facilement modifiables pour les futures composants qui seront choisis par les successeurs de ce projet. Nous avons donc [oled_display.c](#) pour l'écran, [joystick_control.c](#) pour le fait de contrôler la vitesse du moteur avec le joystick, [failsafe.c](#) pour la feature du même nom et enfin [bluetooth.c](#) pour assurer la connexion avec la télécommande.

3.1 - Gestion de l'écran

Commençons par l'affichage sur l'écran. Pour cela, nous utiliserons en réalité trois fichiers différents : [oled_display.c](#), [sh1106.c](#) et [fonts.c](#).

3.1.1 - fonts.c

Concernant ce nouveau fichier, il ne comporte que deux structures et deux (très longues) listes où sont répertoriés les ensembles des lettres, symboles et chiffres de l'alphabet français en hexadécimal permettant d'afficher ces derniers sur l'écran. Voici les deux structures que nous retrouvons dedans (nous nous épargnerons les deux longues listes, dont l'une, par exemple, fait une centaine de lignes à elle seule). Précisons aussi que chacune des structures (et donc des listes) permet deux affichages différents, faisant varier la taille du texte.

```
FontDef Font_7x10 = {
    7, // Largeur
    10, // Hauteur
    Font7x10_Data
};
FontDef Font_11x18 = {
    11,
    18,
    Font11x18_Data
};
```

Figure 43 - Structures pour la police dans deux tailles différentes

3.1.2 - sh1106.c

Passons au deuxième fichier, qui cette fois-ci, est propre à notre composant, à notre écran. C'est donc ce code qui sera à modifier si l'on souhaite changer d'écran (ce dernier est petit : il ne mesure que 1.3pouces). Dans ce dernier, on retrouve encore différentes fonctions qui ont chacune une utilité. Comme tout composant, il nous faut une fonction d'initialisation, que nous appelons ici [SH1106_Init](#).

```

void SH1106_Init(void) {
    HAL_Delay(100); // Attente matériel
    SH1106_WriteCommand(0xAE); // Display OFF
    SH1106_WriteCommand(0xA8); // Set Multiplex Ratio
    SH1106_WriteCommand(0x3F);
    SH1106_WriteCommand(0xD3); // Set Display Offset
    SH1106_WriteCommand(0x00);
    SH1106_WriteCommand(0x40); // Set Display Start Line
    SH1106_WriteCommand(0xA1); // Set Segment Re-map
    SH1106_WriteCommand(0xC8); // Set COM Output Scan Direction
    SH1106_WriteCommand(0xDA); // Set COM Pins Hardware Configuration
    SH1106_WriteCommand(0x12);
    SH1106_WriteCommand(0x81); // Set Contrast Control
    SH1106_WriteCommand(0x7F);
    SH1106_WriteCommand(0xA4); // Entire Display ON
    SH1106_WriteCommand(0xA6); // Set Normal Display
    SH1106_WriteCommand(0xD5); // Set Display Clock Divide Ratio
    SH1106_WriteCommand(0x80);
    SH1106_WriteCommand(0x8D); // Charge Pump Control
    SH1106_WriteCommand(0x14);
    SH1106_WriteCommand(0xAF); // Display ON
    SH1106_Clear();
    SH1106_UpdateScreen();
}

```

Figure 44 - Fonction d'initialisation de l'écran

Et dans cette fonction on retrouve trois nouvelles fonctions que nous avons codé aussi : *SH1106_Clear*, *SH1106_UpdateScreen* et *SH1106_WriteCommand*. Respectivement, et comme leur nom l'annonce, permet d'écrire une commande pour l'écran, d'enlever l'ensemble des pixels sur l'écran et mettre à jour ce dernier (c'est à dire faire en sorte que ce dernier soit en attente d'un affichage).

```

void SH1106_WriteCommand(uint8_t command) {
    HAL_I2C_Mem_Write(&hi2c1, SH1106_I2C_ADDR, 0x00, 1, &command, 1, 10);
}

void SH1106_Clear(void) {
    memset(SH1106_Buffer, 0, sizeof(SH1106_Buffer));
}

void SH1106_UpdateScreen(void) {
    for (uint8_t i = 0; i < 8; i++) {
        SH1106_WriteCommand(0xB0 + i);

        SH1106_WriteCommand(0x00); // Adresse de la colonne droite : 2
        SH1106_WriteCommand(0x10); // Adresse de la colonne gauche : 0

        HAL_I2C_Mem_Write(&hi2c1, SH1106_I2C_ADDR, 0x40, 1,
                          &SH1106_Buffer[SH1106_WIDTH * i], SH1106_WIDTH, 25);
    }
}

```

Figure 45 - Fonctions principales de notre écran

Bien évidemment, nous avons d'autres fonctions dans ce fichier, mais nous ne les détaillerons pas ici. Entre autres, nous avons des fonctions pour dessiner des rectangles vides et pleins, une autre pour colorier les pixels et enfin une gérant le système de coordonnées.

3.1.3 - oled_display.c

Et enfin, dernier fichier, ne permettant qu'une chose : choisir ce qu'on affiche sur notre écran. Pour cela, nous n'avons donc qu'une fonction : **OLED_update** qui s'occupe de faire cela.

```
void OLED_Update(uint8_t isArmed, uint32_t motorSpeedPercent, uint8_t isConnected) {
    SH1106_Clear();

    // 1. Affichage de la connexion (en haut à droite ou ligne dédiée)
    SH1106_GotoXY(0, 0);
    if (isConnected) {
        SH1106_Puts("BT: OK", &Font_7x10, 1);
    } else {
        SH1106_Puts("BT: DISCONNECTED", &Font_7x10, 1);
    }

    // 2. Affichage de l'état (décalé un peu plus bas)
    SH1106_GotoXY(0, 12);
    if (isArmed) {
        SH1106_Puts("STATUS: ARMED", &Font_7x10, 1);
    } else {
        SH1106_Puts("STATUS: LOCKED", &Font_7x10, 1);
    }

    // 3. Affichage de la vitesse
    SH1106_GotoXY(0, 25);
    char buffer[20];
    sprintf(buffer, "SPEED: %lu%%", motorSpeedPercent);
    SH1106_Puts(buffer, &Font_11x18, 1);

    // 4. Dessin d'une barre de progression
    SH1106_DrawRectangle(0, 50, 127, 10, 1);
    uint32_t barWidth = (motorSpeedPercent * 127) / 100;
    if (barWidth > 127) barWidth = 127;
    SH1106_DrawFilledRectangle(0, 50, barWidth, 10, 1);
    SH1106_UpdateScreen();
}
```

Figure 46 - Choix de ce que l'on affiche sur l'écran

3.2 - Joystick et moteur

Passons maintenant à la partie contrôle du moteur à l'aide d'un joystick et du fichier **joystick_control.c** en particulier. Ce fichier d'environ deux cent lignes comporte tout un tas de fonctions : une permettant de gérer la machine à état d'armement du moteur, des fonctions permettant de faire sonner le buzzer quand nécessaire mais surtout une pour contrôler le moteur.

3.2.1 - Buzzers

Ici nous voulons jouer un son à deux moments opposés : lorsque le drone devient prêt à l'utilisation et lorsqu'il ne l'est plus. Nous avons donc, pour cela, deux fonctions (car deux sonorités différentes) : **Sound_Armed** et **Sound_Disarmed**.

```
static void Sound_Armed(void)    { JC_Buzzer_Play(600, 2000); }
static void Sound_Disarmed(void) { JC_Buzzer_Play(600, 8000); }
```

Figure 47 - Alerte sonore selon l'état de commandabilité du drone

Et l'on aperçoit que dans ces fonctions que nous en retrouvons une autre, prenant comme paramètre la durée du son jouer et le pitch soit la "sonorité". Voici cette fonction plus en détail :

```
void JC_Buzzer_Play(uint32_t duration_ms, uint32_t pitch_delay_us) {
    uint32_t start = HAL_GetTick();
    while((HAL_GetTick() - start) < duration_ms) {
        HAL_GPIO_TogglePin(BUZZER_GPIO_Port, BUZZER_Pin);
        /* Boucle de délai logiciel pour ajuster la tonalité */
        for(volatile int i=0; i < pitch_delay_us; i++);
    }
    /* Assure que la sortie est à l'état bas après le bip */
    HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin, GPIO_PIN_RESET);
}
```

Figure 48 - Envoi de la commande au buzzer

Et c'est donc avec cela que nous arrêtons tout ce qui concerne le buzzer, du moins pour la partie moteur.

3.2.2 - Contrôle du moteur

Nous pouvons découper cette nouvelle sous-partie en deux parties : une concernant l'état d'armement et une pour le contrôle directe du moteur. Concernant la première, nous avons deux fonctions : *JC_Handle_Arming* et *JC_Force_Disarm* (qui nous servira dans le prochain fichier).

```
void JC_Handle_Arming(void) {
    static uint32_t pressStartTime = 0;
    static uint8_t buttonPressed = 0;
    static uint8_t actionDone = 0;
    /* Lecture de l'état du bouton */
    if (HAL_GPIO_ReadPin(JS_BUTTON_GPIO_Port, JS_BUTTON_Pin) == GPIO_PIN_RESET)
    {
        if (!buttonPressed) {
            buttonPressed = 1;
            pressStartTime = HAL_GetTick();
            actionDone = 0;
        }
        /* Vérification du seuil de temps pour l'appui long */
        if (!actionDone && (HAL_GetTick() - pressStartTime >= ARM_TIME_MS)) {
            isArmed = !isArmed; // Bascule de l'état de sécurité
            if (isArmed) {
                Sound_Armed();
            } else {
                Sound_Disarmed();
            }
            actionDone = 1; // Marqueur pour éviter de répéter l'action
        }
    } else {
        /* Réinitialisation lors du relâchement du bouton */
        buttonPressed = 0;
        actionDone = 0;
    }
}

void JC_Force_Disarm(void) {
    isArmed = 0;
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);
    OLED_Update(0, 0);
}
```

Figure 49 - Armement ou désarmement du moteur à l'aide du joystick

```

void JC_Control_Motor(void) {
    uint32_t joy_val = 0;
    uint32_t current_speed = 0;

    const uint32_t VAL_BAS      = 0;
    const uint32_t VAL_MILIEU   = 2730; // Position neutre du joystick
    const uint32_t VAL_HAUT     = 4095;
    const uint32_t DEADZONE     = 50;   // Petite marge pour stabiliser le milieu
    HAL_ADC_Start(&hadc2);
    if (HAL_ADC_PollForConversion(&hadc2, 10) == HAL_OK) {
        joy_val = HAL_ADC_GetValue(&hadc2);
    }
    HAL_ADC_Stop(&hadc2);
    if (isArmed) {
        // ZONE 1 : Du bas vers le milieu
        if (joy_val < (VAL_MILIEU - DEADZONE)) {
            // Mappage VAL_BAS à VAL_MILIEU -> 0 à PWM_MAX/2
            current_speed = (joy_val * (PWM_MAX_VALUE / 2)) / (VAL_MILIEU - VAL_BAS);
        }
        // ZONE MORTE : On force le 50% quand on est proche du milieu
        else if (joy_val >= (VAL_MILIEU - DEADZONE) && joy_val <= (VAL_MILIEU + DEADZONE)) {
            current_speed = PWM_MAX_VALUE / 2;
        }
        // ZONE 2 : Du milieu vers le haut
        else {
            // Mappage VAL_MILIEU à VAL_HAUT -> PWM_MAX/2 à PWM_MAX
            uint32_t input_offset = joy_val - VAL_MILIEU;
            uint32_t input_range  = VAL_HAUT - VAL_MILIEU;
            uint32_t output_start = PWM_MAX_VALUE / 2;
            uint32_t output_range = PWM_MAX_VALUE / 2;
            current_speed = output_start + (input_offset * output_range) / input_range;
        }

        // Sécurité finale
        if (current_speed > PWM_MAX_VALUE) current_speed = PWM_MAX_VALUE;
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, current_speed);
    } else {
        current_speed = 0;
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);
    }

    uint8_t connected = (HAL_GPIO_ReadPin(BT_STATE_GPIO_Port, BT_STATE_Pin) == GPIO_PIN_SET);
    uint32_t speed_percent = (current_speed * 100) / PWM_MAX_VALUE;
    OLED_Update(isArmed, speed_percent, connected);
}

```

Figure 50 - Contrôle du moteur à l'aide du joystick

C'est donc cette fonction que l'on retrouvera en boucle dans le `main.c`. Notons ici les deux mappages que l'on retrouve selon si le joystick se retrouve au-dessus de sa position neutre ou en-dessous.

Il ne nous reste donc plus que deux fichiers à voir, celui concernant le module bluetooth permettant l'échange de données mais aussi celui concernant le failsafe permettant un arrêt en sécurité en cas de problème.

3.2.3 - Échange de données en bluetooth

Pour ce nouveau module, nous utiliserons le HM10RF BT, un module de transmission Bluetooth. Cependant, n'ayant pas de module de réception, nous connecterons donc notre téléphone à ce dernier et pourrions observer les données que l'on veut dessus.

Dans ce nouveau fichier on retrouve deux fonctions : celle d'initialisation habituelle **BT_Init_Loopback**, ainsi que celle pour envoyer les données **BT_Process_Data**. Pour l'initialisation nous avons la même chose qu'habituellement, nous envoyons juste certaines données à notre module et attendons la connexion. Une fois celle-ci effectuée nous en profitons pour afficher un message sur la console PuTTY.

```
void BT_Init_Loopback(UART_HandleTypeDef *huart_hm10, UART_HandleTypeDef
*huart_hc06, UART_HandleTypeDef *huart_putty) {
    hm10_uart = huart_hm10;
    pc_uart    = huart_putty;
    HAL_Delay(500);

    HAL_UART_Receive_IT(hm10_uart, &rx_byte_hm10, 1);
    HAL_UART_Receive_IT(pc_uart, &rx_byte_pc, 1);
    HAL_UART_Transmit(pc_uart, (uint8_t*)"Liaison PC-BT Active\r\n", 22, 100);
}
```

Figure 51 - Initialisation du module

Désormais initialisé nous retrouvons la fonction permettant d'envoyer des données par bluetooth à l'appareil connecté.

```
void BT_Process_Data(UART_HandleTypeDef *huart) {
    if (huart->Instance == hm10_uart->Instance) {

        HAL_UART_Transmit(pc_uart, &rx_byte_hm10, 1, 10);

        if (rx_byte_hm10 == '\r' || rx_byte_hm10 == '\n') {
            uint8_t nl[] = "\n";
            HAL_UART_Transmit(pc_uart, nl, 1, 10);
        }
        HAL_UART_Receive_IT(hm10_uart, &rx_byte_hm10, 1);
    }
    else if (huart->Instance == pc_uart->Instance) {
        // On stocke l'octet reçu dans le tableau
        tx_buffer[head] = rx_byte_pc;
        head = (head + 1) % TX_BUFFER_SIZE;
        // Relance de l'écoute pc
        HAL_UART_Receive_IT(pc_uart, &rx_byte_pc, 1);
    }
}
```

Figure 52 - Envoi de données par bluetooth

Voici ainsi ce que nous obtenons, à la fois sur la liaison série mais aussi sur le téléphone connecté en bluetooth sur lequel on envoie donc le pourcentage de la vitesse :

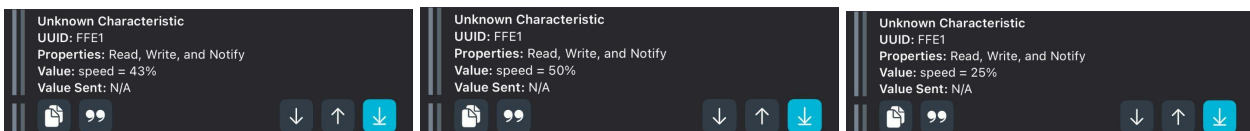


Figure 53 - Module bluetooth opérationnel

Il ne nous reste donc plus que notre feature de sécurité puisque maintenant nous avons tout le reste.

3.2.4 - Feature de sécurité

Les différentes conditions pour avoir à faire à cette feature, nous ne les répéterons donc pas. Une fois de plus, nous retrouvons une initialisation plus simple où nous vérifions simplement que tout est bel et bien éteint au démarrage (par tout nous entendons un buzzer et une LED qui feront du bruit et clignotteront lors d'un problème).

```
void FAILSAFE_Init(void) {
    HAL_GPIO_WritePin(BT_LED_GPIO_Port, BT_LED_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(BT_BUZZ_GPIO_Port, BT_BUZZ_Pin, GPIO_PIN_RESET);
}
```

Figure 54 - Initialisation du Failsafe

Avant tout, créons une nouvelle structure dans notre failsafe.h, une structure ne comportant que deux booléens : un pour la sécurité du système et un pour l'état de la connexion. Cette liste sera amené à être augmenté selon les features de sécurité que nous rajouterons (température, courant, batterie, etc.).

```
typedef enum {
    SYSTEM_SAFE = 0,
    SYSTEM_LOST = 1
} Failsafe_State_t;
```

Figure 55 - Structure pour le Failsafe

Codons désormais comment l'utilisateur sera au courant à l'aide de la fonction Failsafe_Buzzer_Play. Dans ces fonctions nous avons un buzzer qui buzze tant que le problème n'est pas réglé (ou tant que le drone n'est pas retourné à sa position initiale). En plus, une LED clignottera, mais ceci se trouve directement dans la fonction principale.

```
void Failsafe_Buzzer_Play(uint32_t duration_ms, uint32_t pitch_delay_us) {
    uint32_t start = HAL_GetTick();
    while((HAL_GetTick() - start) < duration_ms) {
        HAL_GPIO_TogglePin(BT_BUZZ_GPIO_Port, BT_BUZZ_Pin);
        for(volatile int i=0; i < pitch_delay_us; i++);
    }
    HAL_GPIO_WritePin(BT_BUZZ_GPIO_Port, BT_BUZZ_Pin, GPIO_PIN_RESET);
}
```

Figure 56 - Son du buzzer en cas de Failsafe

Il ne nous reste donc plus qu'à proposer notre fonction principale : **FAILSAFE_CheckConnection**. Nous ajoutons d'ailleurs une sécurité dans notre sécurité : si nous perdons le signal plus de 200ms alors nous activons l'événement, si non, nous ne ferons rien (il arrive de perdre la connexion pendant un très petit laps de temps).

```

Failsafe_State_t FAILSAFE_CheckConnection(void) {
    static uint32_t lastStableTime = 0;
    static GPIO_PinState lastState = GPIO_PIN_RESET;
    GPIO_PinState currentState = HAL_GPIO_ReadPin(BT_STATE_GPIO_Port,
BT_STATE_Pin);
    // Si l'état change, on réinitialise le compteur
    if (currentState != lastState) {
        lastStableTime = HAL_GetTick();
        lastState = currentState;
    }
    // On ne valide le changement que s'il est stable depuis plus de 200ms
    if ((HAL_GetTick() - lastStableTime) > 200) {
        if (currentState == GPIO_PIN_RESET) {
            // --- DÉCONNECTÉ (STABLE) ---
            HAL_GPIO_TogglePin(BT_LED_GPIO_Port, BT_LED_Pin);
            Failsafe_Buzzer_Play(1000, 10000);
            return SYSTEM_LOST;
        }
    }
    // --- CONNECTÉ (OU TRANSITION) ---
    HAL_GPIO_WritePin(BT_LED_GPIO_Port, BT_LED_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(BT_BUZZ_GPIO_Port, BT_BUZZ_Pin, GPIO_PIN_RESET);
    return SYSTEM_SAFE;
}

```

Figure 57 - Fonction principale de notre Failsafe

η - Conclusion

Finalement, qu'avons-nous proposé pour ce projet et qu'en avons-nous appris ? Au niveau du résultat obtenu, nous finissons avec rien de bien concret : aucun véhicule volant ou planant. Mais ceci s'explique par la complexité du projet et le peu de temps qui nous était accordé. Cependant, plein de modules séparés sont prêts à être mis en commun et/ou à être adaptés. Physiquement, nous n'avons donc pas grand chose à proposer (si ce n'est le contrôle des servomoteurs avec la manette ainsi que le montage sur la breadboard) mais un ensemble de lignes de code prêtes pour les personnes qui aimeraient reprendre ce projet (disponible sur notre github).

Pour ce que l'on a appris, c'est dans un premier temps de ne pas avoir les yeux plus gros que le ventre, vouloir participer au Dassault UAV Challenge avec une limite de temps si importante n'est pas envisageable (à moins de négliger les cours). Ensuite les compétences sont variées selon les postes : certains ont développé des compétences en Python, d'autres en management mais aussi en asservissement. Ce qui est sûr c'est que nous avons tous les trois développé notre compétence en codage en langage C et nos compétences humaines: le travail en équipe ainsi que la communication.